Subscription Architecture — Appstle + Shopify + Collector’s Cabinet

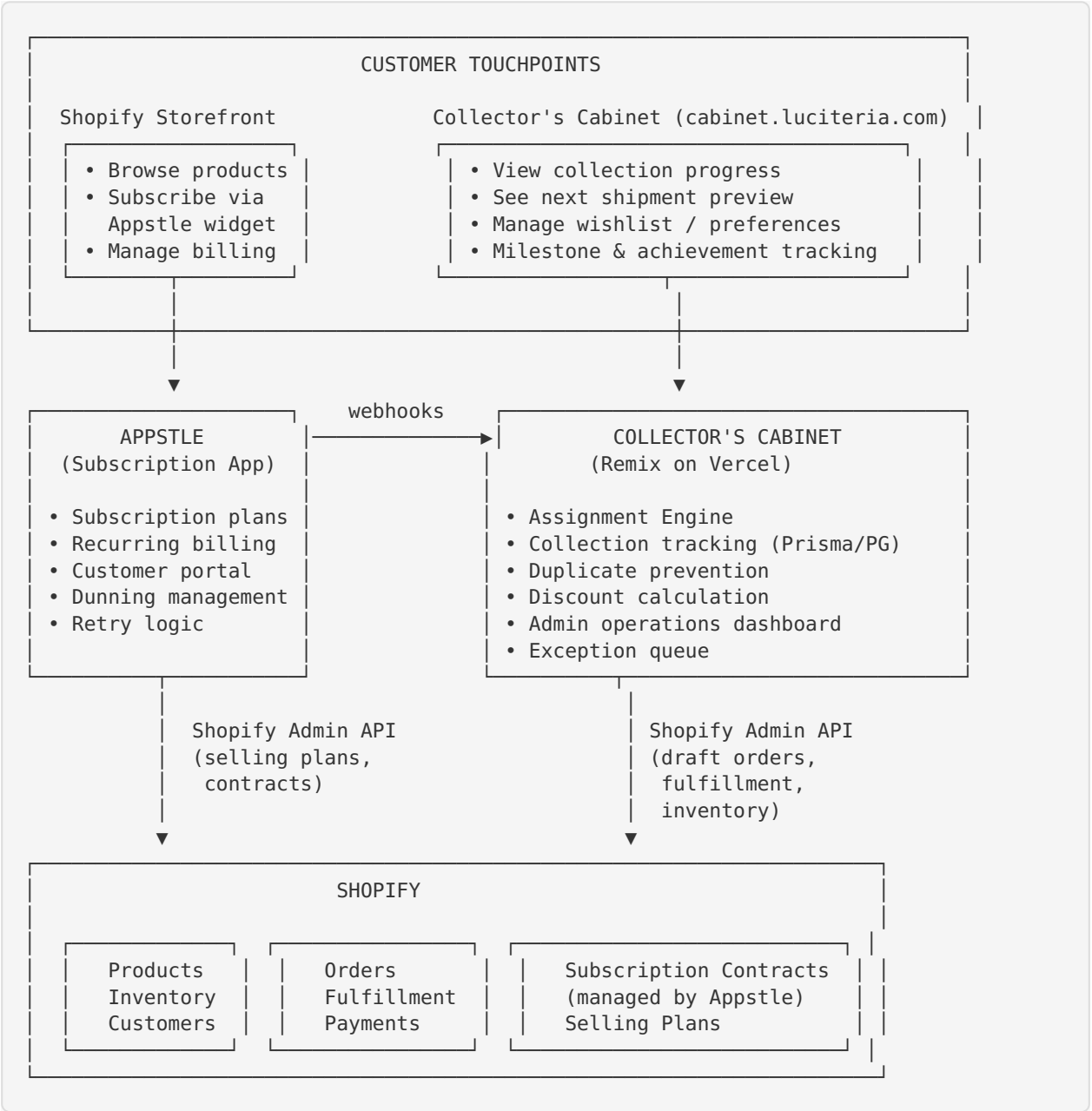
> **Status:** Implementation Blueprint
> **Last Updated:** 2026-07-15
> **Stack:** Appstle (subscription billing) → Shopify (ecommerce/orders) → Collector’s Cabinet (Remix/Vercel/PostgreSQL)

Table of Contents

---

1. System Overview

Architecture at a Glance



Responsibilities by System

System	Owns	Does NOT Own
Appstle	Subscription billing lifecycle, payment retry/dunning, selling plan creation, customer-facing subscription portal, plan swap/pause/cancel UI	Product assignment, collection tracking, which product ships next
Shopify	Product catalog, inventory levels, order creation & fulfillment, payment processing, customer records	Assignment logic, subscription business rules, collection state
Cabinet	Assignment engine, collection tracking, duplicate prevention, precious metal exclusion, discount calculations, admin operations, exception queue	Payment processing, billing retries, storefront UI

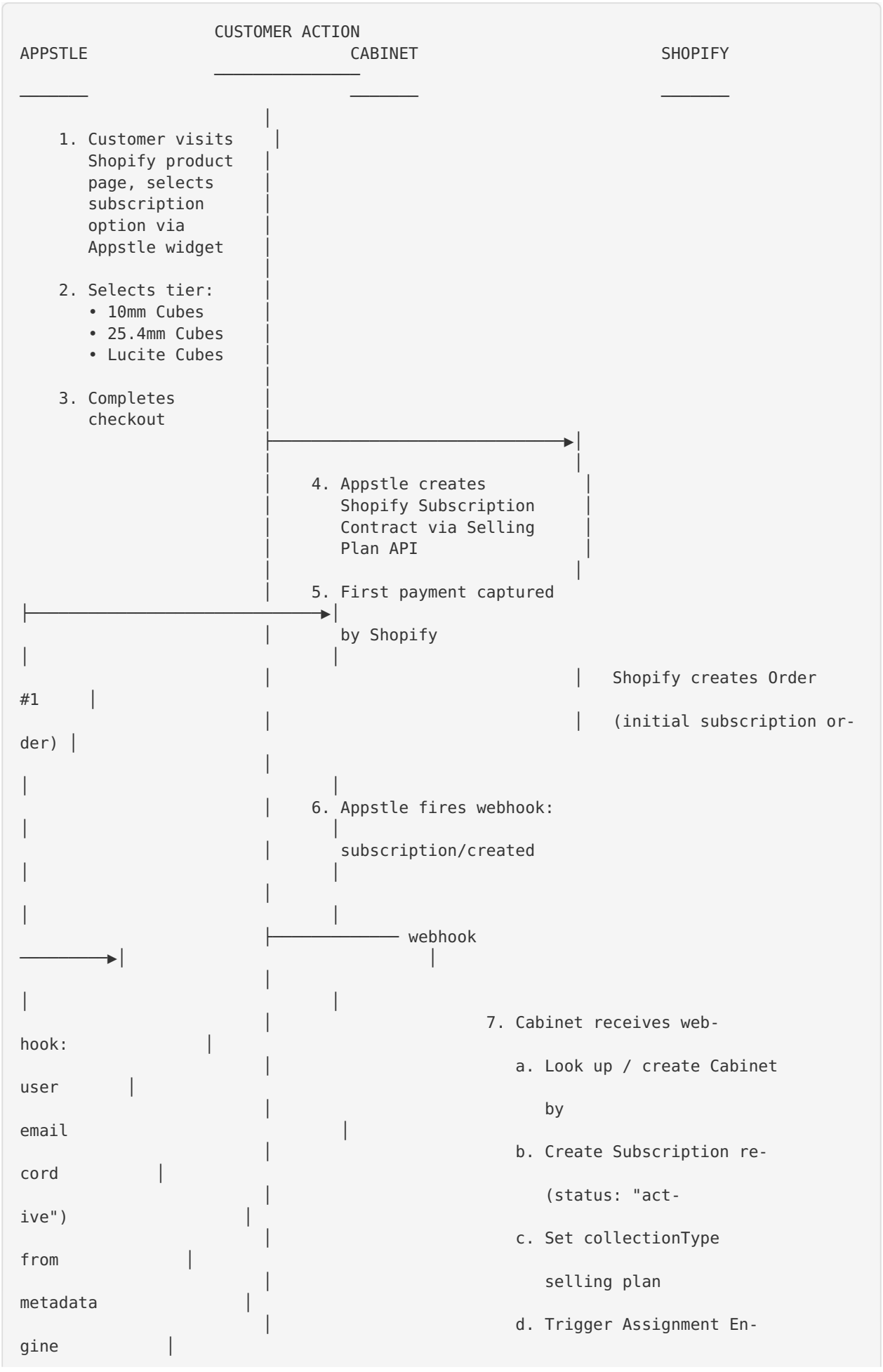
Key Design Principles

1. **Appstle is the billing brain** — it handles all recurring charge logic, dunning, retries, and the customer self-service portal for managing subscriptions.
2. **Cabinet is the assignment brain** — when Appstle says “this customer renewed,” Cabinet decides which product ships and creates the Shopify order.
3. **Shopify is the fulfillment backbone** — all orders flow through Shopify for payment capture, inventory decrement, and shipping label generation.
4. **Email is the identity bridge** — Shopify customer email = Cabinet user email. No OAuth required initially.
5. **Webhooks are the nervous system** — Appstle → Cabinet webhooks trigger all automated logic.

---

2. Data Flow Diagrams

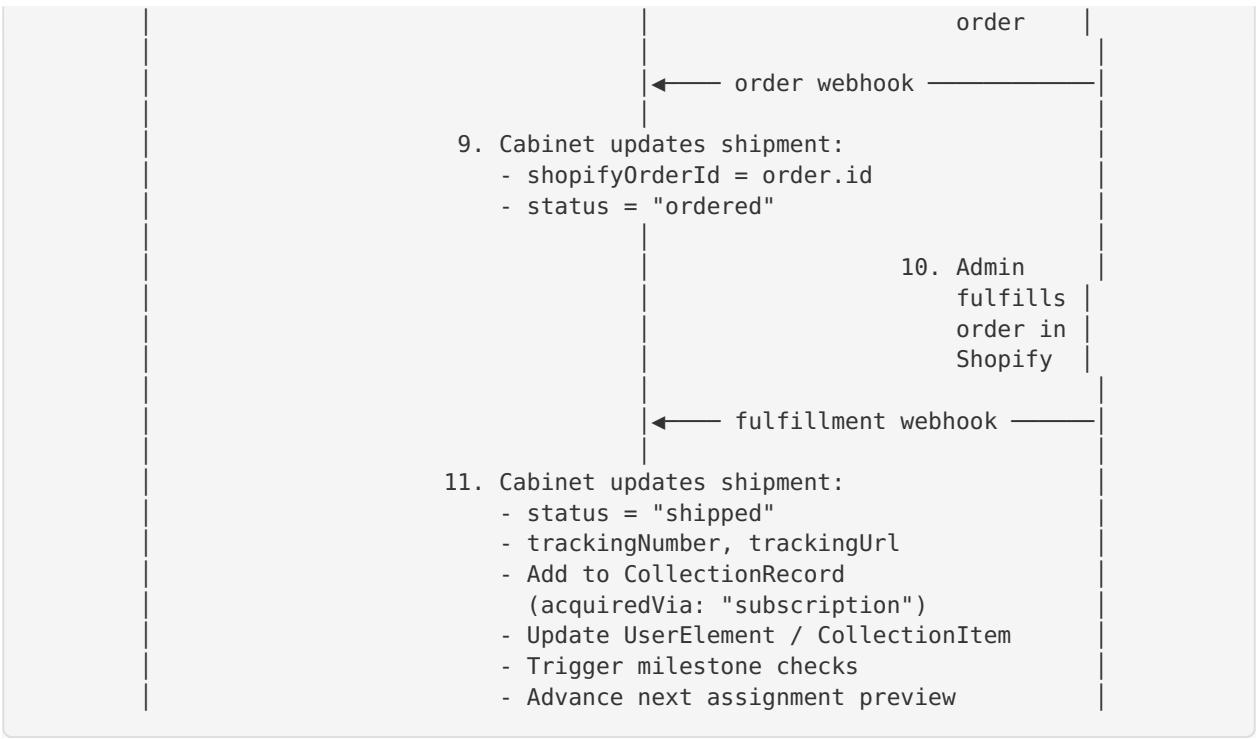
2.1 New Subscription Signup Flow


CUSTOMER ACTION
APPSTLE
CABINET
SHOPIFY
1. Customer visits Shopify product page, selects subscription option via Appstle widget
2. Selects tier:
• 10mm Cubes
• 25.4mm Cubes
• Lucite Cubes
3. Completes checkout
4. Appstle creates Shopify Subscription Contract via Selling Plan API
5. First payment captured by Shopify
Shopify creates Order #1 (initial subscription order)
6. Appstle fires webhook: subscription/created
webhook
7. Cabinet receives webhook:
a. Look up / create Cabinet user by email
b. Create Subscription record (status: "active")
c. Set collectionType from selling plan metadata
d. Trigger Assignment Engine

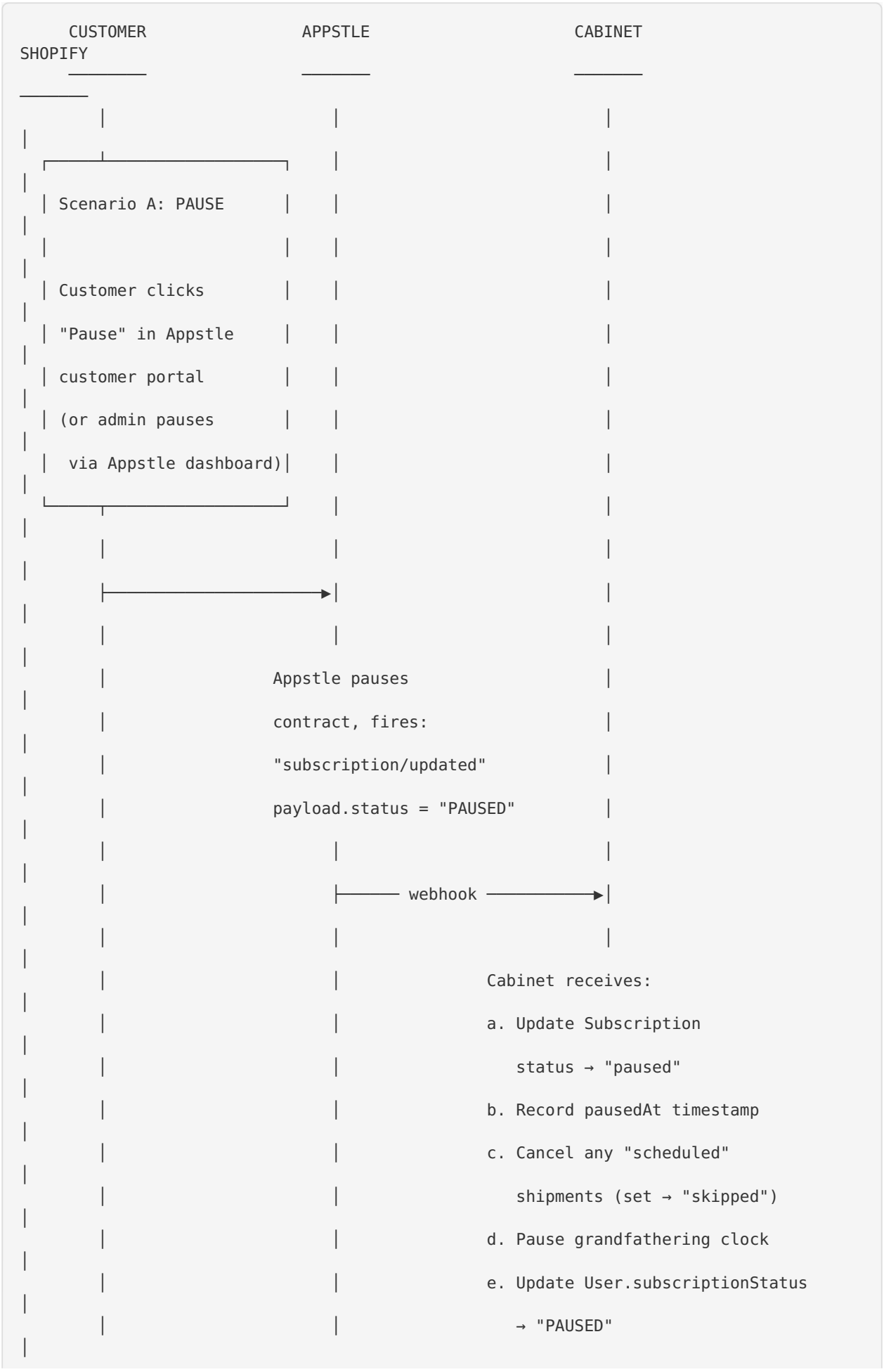
```
                          |                              for FIRST SHIP-
MENT                    |
|                                                                  |
                          |                     8. Assignment Engine
runs:                  |
                          |                        a. Build candidate
pool                  |
                          |                           (collection type, in-
stock,          |
                          |                           exclude precious
metals)            |
                          |                        b. Filter: not already
owned           |
                          |                        c. Rank by
strategy                    |
                          |                           (default: WISH-
LIST_PRIORITY)         |
                          |                        d. Calculate discount
%                |
                          |                        e. If discount > 20%:
flag             |
                          |                           for admin re-
view                     |
                          |                        f. Create ShipmentItem re-
cord           |
|                                                                  |
                          |                     9. If auto-approved (discount ≤
20%):     |
                          |                        Create Shopify Draft Order
————————————▶|
                          |                        with assigned product vari-
ant           |
|                                                                  |
                          |                    10. If flagged: add
to                      |
                          |                        admin exception
queue                   |
                          |                        (admin approves →
then                   |
                          |                        Shopify order
created)                  |
```

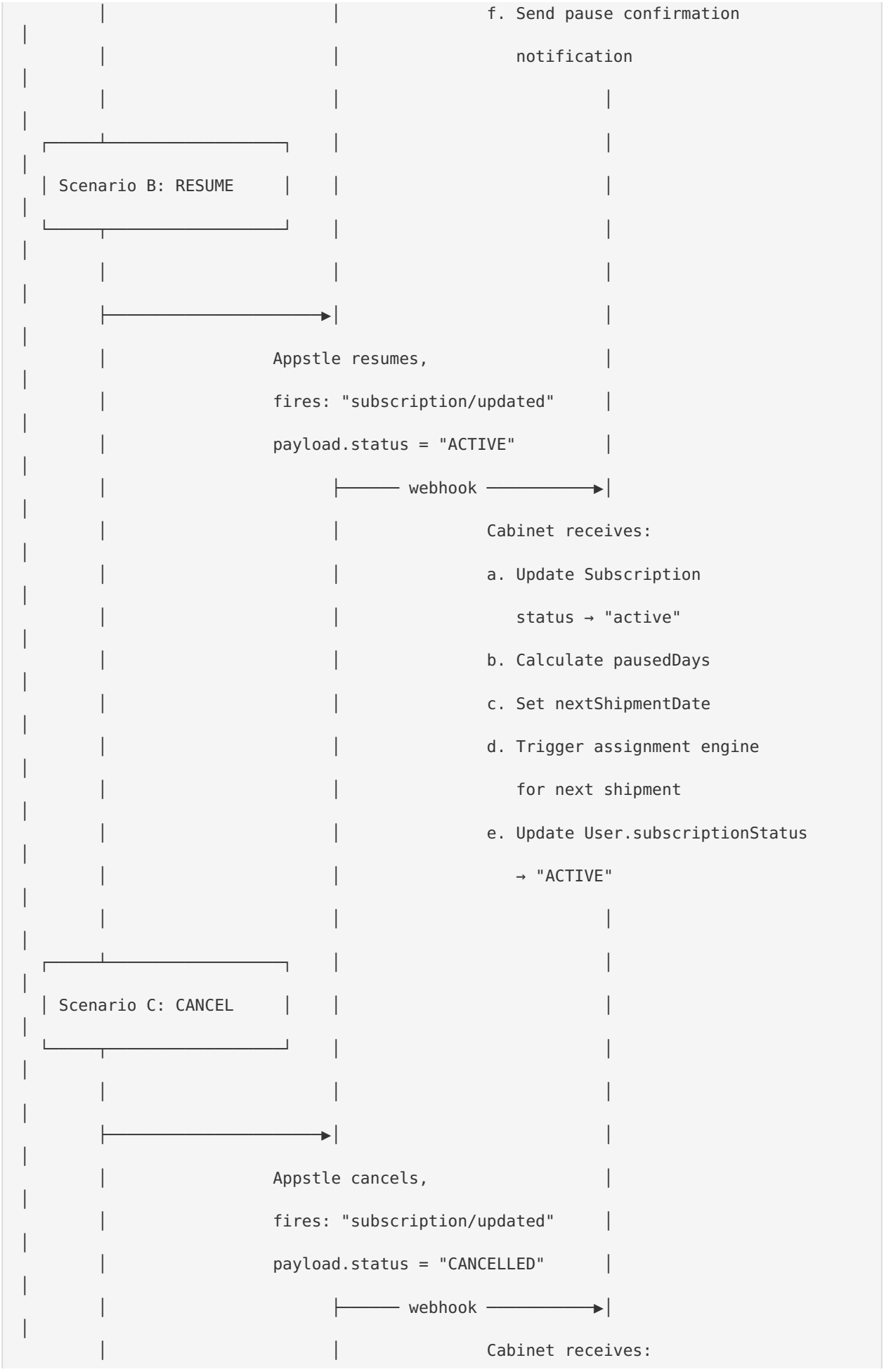
2.2 Monthly Renewal & Product Assignment Flow

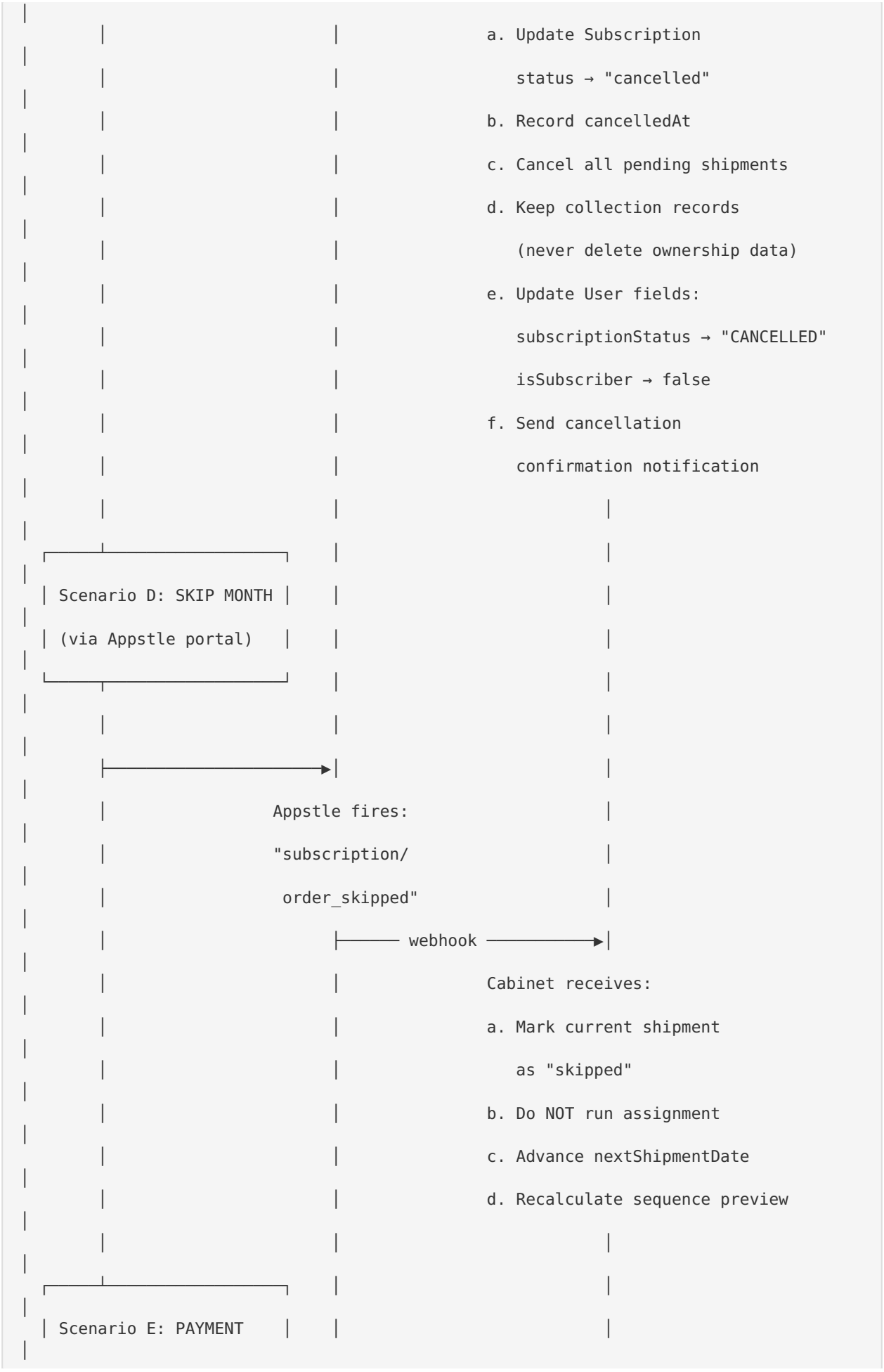

APPSTLE
CABINET
SHOPIFY
1. Appstle billing cycle fires (monthly/quarterly)
2. Appstle charges customer via Shopify Payments
3. Payment succeeds: Appstle fires webhook "subscription_billing_attempt/succeeded"
webhook
4. Cabinet receives billing success:
a. Verify webhook signature
b. Look up Subscription by appstleContractId
c. Update subscription status
d. Log billing event
5. Trigger Assignment Engine:
a. Fetch customer's collection records (owned elements)
b. Fetch previous shipments (shipped product IDs)
c. Apply collection type filter (10mm/25.4mm/lucite)
d. Exclude precious metals (Re,Rh,Au,Os,Ru,Pd,Ir,Pt)
e. Exclude already-owned
f. Apply customer preferences (excluded categories, etc.)
g. Rank candidates by strategy
h. Select top candidate
i. Calculate discount %
j. Create SubscriptionShipment record (status: "assigned")
6. Discount check:
IF discount ≤ 20%:
Auto-approve
IF 20% < discount ≤ 35%:
Route to admin exception queue
IF discount > 35%:
Hard-reject, admin must pick manually
7. On approval (auto or admin):
Create Shopify Draft Order
- Customer email
- Assigned variant ID
- Subscription price
- Tag: "luciteria-subscription"
Draft Order created
8. Shopify converts draft to

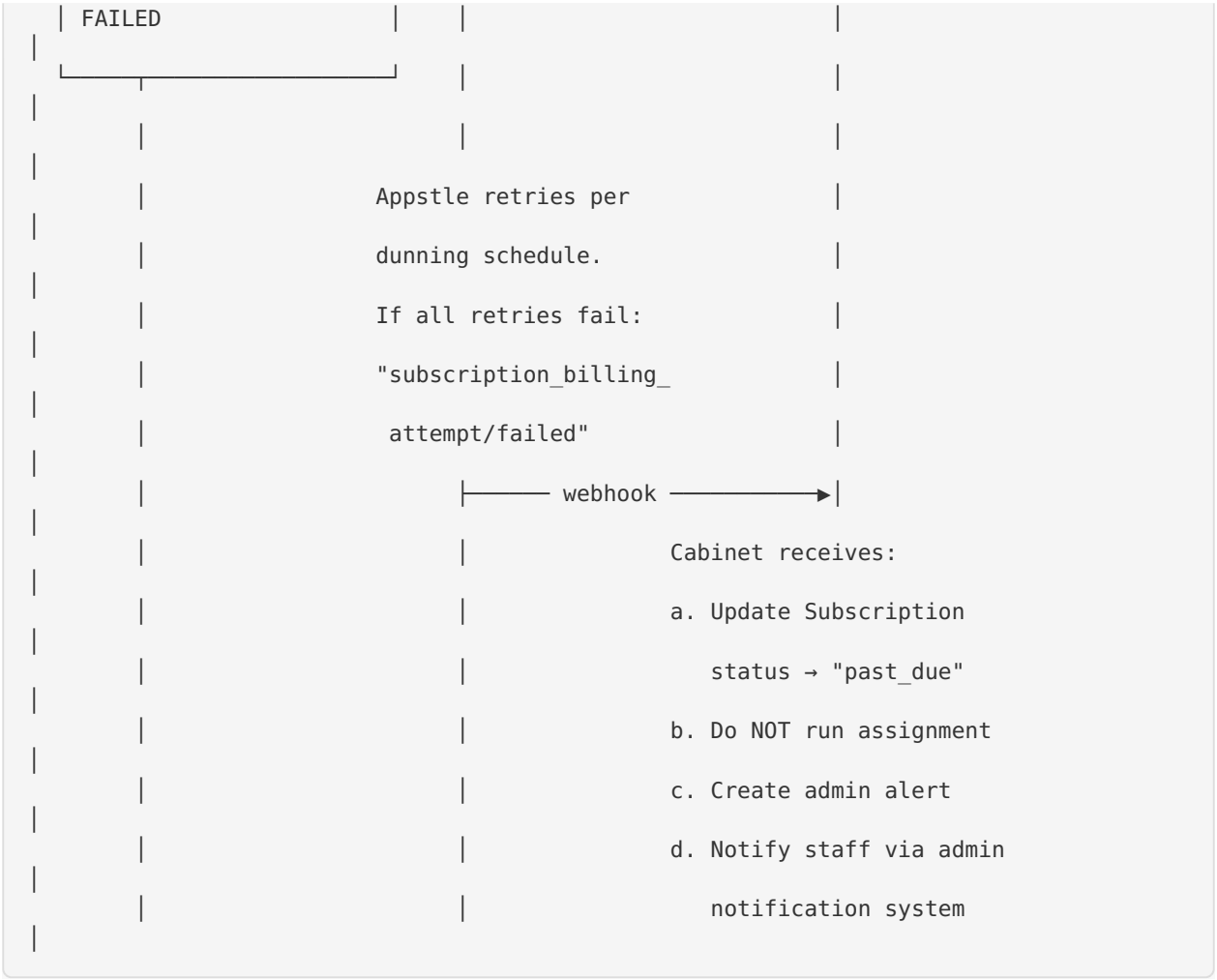

order
order webhook
9. Cabinet updates shipment:
- shopifyOrderId = order.id
- status = "ordered"
10. Admin fulfills order in Shopify
fulfillment webhook
11. Cabinet updates shipment:
- status = "shipped"
- trackingNumber, trackingUrl
- Add to CollectionRecord
(acquiredVia: "subscription")
- Update UserElement / CollectionItem
- Trigger milestone checks
- Advance next assignment preview

2.3 Cancellation / Pause / Skip Flow


CUSTOMER
APPSTLE
CABINET
SHOPIFY
Scenario A: PAUSE
Customer clicks
"Pause" in Appstle
customer portal
(or admin pauses
via Appstle dashboard)
Appstle pauses
contract, fires:
"subscription/updated"
payload.status = "PAUSED"
webhook
Cabinet receives:
a. Update Subscription
status → "paused"
b. Record pausedAt timestamp
c. Cancel any "scheduled"
shipments (set → "skipped")
d. Pause grandfathering clock
e. Update User.subscriptionStatus
→ "PAUSED"


f. Send pause confirmation
notification
Scenario B: RESUME
Appstle resumes,
fires: "subscription/updated"
payload.status = "ACTIVE"
webhook
Cabinet receives:
a. Update Subscription
status → "active"
b. Calculate pausedDays
c. Set nextShipmentDate
d. Trigger assignment engine
for next shipment
e. Update User.subscriptionStatus
→ "ACTIVE"
Scenario C: CANCEL
Appstle cancels,
fires: "subscription/updated"
payload.status = "CANCELLED"
webhook
Cabinet receives:

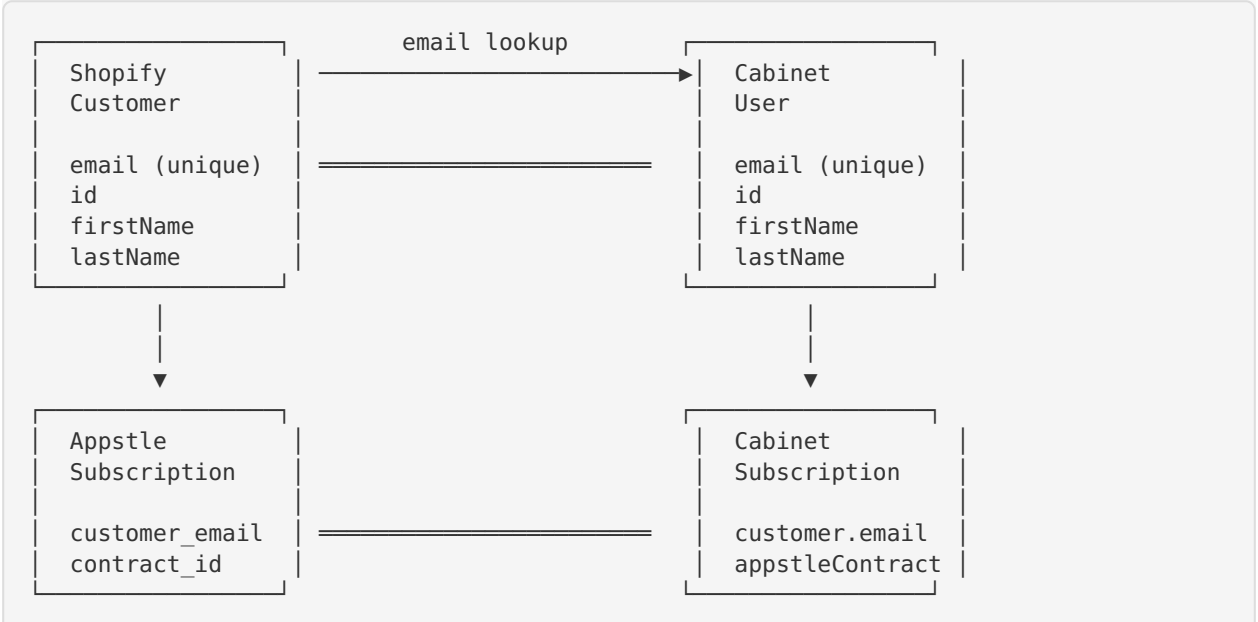

a. Update Subscription
status → "cancelled"
b. Record cancelledAt
c. Cancel all pending shipments
d. Keep collection records
(never delete ownership data)
e. Update User fields:
subscriptionStatus → "CANCELLED"
isSubscriber → false
f. Send cancellation
confirmation notification
Scenario D: SKIP MONTH
(via Appstle portal)
Appstle fires:
"subscription/
order_skipped"
webhook
Cabinet receives:
a. Mark current shipment
as "skipped"
b. Do NOT run assignment
c. Advance nextShipmentDate
d. Recalculate sequence preview
Scenario E: PAYMENT



---

3. Customer Identity Matching

Phase 1: Email-Based Matching (Launch)



Matching Algorithm (on every Appstle webhook):

```
async function resolveCustomer(webhookPayload) {
  const email = webhookPayload.customer_email?.toLowerCase().trim();
  if (!email) throw new Error("No customer email in webhook payload");

  // 1. Try to find existing Cabinet User by email
  let user = await prisma.user.findUnique({ where: { email } });

  // 2. Try to find existing Customer record by email
  let customer = await prisma.customer.findUnique({ where: { email } });

  // 3. If neither exists, create both
  if (!user) {
    user = await prisma.user.create({
      data: {
        email,
        firstName: webhookPayload.customer_first_name || "",
        lastName: webhookPayload.customer_last_name || "",
        passwordHash: "APPSTLE_MANAGED", // No password — managed externally
        wishlistToken: crypto.randomUUID(),
        userType: "subscriber",
        isSubscriber: true,
        subscriptionStatus: "ACTIVE",
        onboardingStep: 1, // Will need to complete Cabinet onboarding
      },
    });
  }

  if (!customer) {
    customer = await prisma.customer.create({
      data: {
        email,
        firstName: webhookPayload.customer_first_name || "",
        lastName: webhookPayload.customer_last_name || "",
        displayName: webhookPayload.customer_first_name || "Collector",
        shopifyCustomerId: webhookPayload.shopify_customer_id?.toString(),
        collectionType: mapSellingPlanToCollectionType(webhookPayload),
      },
    });
  }

  return { user, customer };
}
```

Edge Cases:

Scenario	Handling
Customer exists in Cabinet but not subscribed	Update `isSubscriber=true`, `subscriptionStatus="ACTIVE"`
Customer exists in Shopify but not in Cabinet	Create Cabinet User + Customer on first webhook
Email change in Shopify	`customers/update` webhook updates Cabinet email (cascade to User + Customer)
Duplicate emails (impossible)	Both Shopify and Cabinet enforce unique email — conflict returns 409
Customer has Cabinet account with password	Keep existing account, add subscription fields — they can still log in with password

Phase 2: OAuth Upgrade Path (Future)

```
┌──────────────────────────────────────────────────────────────┐
│  FUTURE: Shopify Multipass or OAuth SSO                      │
│                                                              │
│  1. Customer logs into Shopify storefront                    │
│  2. Shopify issues Multipass token with customer ID + email  │
│  3. Cabinet validates Multipass token                        │
│  4. Cabinet creates session tied to Shopify customer ID      │
│  5. No separate Cabinet password needed                      │
│                                                              │
│  Benefits:                                                   │
│  - Single sign-on across storefront + Cabinet                │
│  - Shopify customer ID becomes the canonical identifier      │
│  - Email changes automatically propagated                    │
│  - Works with Shopify's existing auth infrastructure         │
│                                                              │
│  Migration:                                                  │
│  - Add shopifyCustomerId to User model (already on Customer) │
│  - Backfill from Customer.shopifyCustomerId → User           │
│  - Remove passwordHash requirement (keep for admin users)    │
│  - Add Multipass validation middleware                       │
│                                                              │
│  Required: Shopify Plus plan (Multipass is Plus-only)        │
│  Alternative: Shopify OAuth via custom app (any plan)        │
└──────────────────────────────────────────────────────────────┘
```

---

4. Webhook Events

4.1 Appstle Webhook Events

Appstle sends webhooks to a configurable endpoint. Our endpoint: `https://cabinet.luciteria.com/webhooks/appstle`

Webhook Event	Trigger	Cabinet Action	Priority
`subscription/created`	New subscription contract created	Create Subscription + Customer records, trigger first assignment	**P0**
`subscription/updated`	Status change (pause/resume/cancel/plan swap)	Update Subscription status, handle pause/resume/cancel logic	**P0**
`subscription_billing_attempt/succeeded`	Monthly payment captured successfully	Trigger assignment engine for this billing cycle	**P0**
`subscription_billing_attempt/failed`	Payment failed (after all Appstle retries exhausted)	Mark subscription `past_due`, alert admin, skip assignment	**P0**
`subscription/order_skipped`	Customer skips upcoming order via Appstle portal	Mark shipment as `skipped`, advance next date	**P1**
`subscription/contract_renewed`	Contract renewed for next billing cycle	Log renewal, ensure subscription is active	**P1**
`subscription/plan_changed`	Customer swaps between tiers (e.g., 10mm → lucite)	Update collectionType, recalculate assignment preview, handle collection type change history	**P1**
`subscription/product_swapped`	Customer requests product swap before shipment	Queue admin review (we don't allow self-service product swaps)	**P2**
`subscription/cancelled`	Subscription fully cancelled	Cancel subscription, clean up scheduled shipments	**P0**

4.2 Shopify Webhook Events (Existing — Augmented for Subscriptions)

These are already partially implemented in `shopify-webhooks.server.js`. Subscription-specific additions noted.

Webhook Topic	Subscription-Specific Action	Status
`products/update`	Recalculate discount % on affected scheduled shipments (already implemented)	✅ Live
`products/delete`	Re-assign any shipments that had this product	🔲 Stub
`inventory_levels/update`	Trigger OOS-shift on assignment preview if product goes to 0 (already implemented)	✅ Live
`orders/create`	Link order to Subscription-Shipment if tagged `luciteria-subscription`	🔲 Stub
`orders/paid`	Update shipment status → "paid"	🔲 Stub
`orders/fulfilled`	Update shipment → "shipped", add to Collection-Record, check milestones	🔲 Stub
`customers/create`	Pre-create Cabinet User/Customer if not exists	🔲 Stub
`customers/update`	Sync email/name changes to Cabinet	🔲 Stub

4.3 Webhook Payload Examples

Appstle `subscription/created` payload (expected shape):

```
{
  "event": "subscription/created",
  "subscription_contract_id": "appstle_contract_12345",
  "shopify_subscription_contract_id": "gid://shopify/SubscriptionContract/67890",
  "customer_email": "collector@example.com",
  "customer_first_name": "Jane",
  "customer_last_name": "Doe",
  "shopify_customer_id": "7654321",
  "status": "ACTIVE",
  "selling_plan_name": "Lucite Cubes — Monthly",
  "selling_plan_id": "gid://shopify/SellingPlan/111222",
  "selling_plan_group_name": "Lucite Cubes Subscription",
  "billing_interval": "MONTH",
  "billing_interval_count": 1,
  "price": "49.99",
  "currency": "USD",
  "next_billing_date": "2026-08-15T00:00:00Z",
  "line_items": [
    {
      "variant_id": "gid://shopify/ProductVariant/99999",
      "quantity": 1,
      "price": "49.99"
    }
  ],
  "created_at": "2026-07-15T12:00:00Z",
  "metadata": {
    "collection_type": "lucite",
    "tier": "lucite_monthly"
  }
}
```

Appstle `subscription_billing_attempt/succeeded` payload:

```
{
  "event": "subscription_billing_attempt/succeeded",
  "subscription_contract_id": "appstle_contract_12345",
  "shopify_subscription_contract_id": "gid://shopify/SubscriptionContract/67890",
  "customer_email": "collector@example.com",
  "shopify_customer_id": "7654321",
  "billing_attempt_id": "attempt_abc123",
  "order_id": "gid://shopify/Order/11223344",
  "order_number": "#LUC-1042",
  "amount_charged": "49.99",
  "currency": "USD",
  "billing_date": "2026-08-15T00:00:00Z",
  "next_billing_date": "2026-09-15T00:00:00Z",
  "status": "SUCCESS"
}
```

---

5. API Integration Points

5.1 Appstle → Cabinet (Inbound Webhooks)

All inbound webhooks hit: `POST /webhooks/appstle`

```
┌──────────────────────────────────────────────────────────────────┐
│  ROUTE: /webhooks/appstle                                        │
│  FILE:  app/routes/webhooks.appstle.jsx (new)                    │
│                                                                  │
│  1. Validate HMAC signature (Appstle webhook secret)             │
│  2. Parse event type from payload                                │
│  3. Log to WebhookEventLog (topic: "appstle/{event}")            │
│  4. Route to handler:                                            │
│                                                                  │
│      ┌──────────────────────────────┬──────────────────────────┐ │
│      │ Event                        │ Handler Function         │ │
│      ├──────────────────────────────┼──────────────────────────┤ │
│      │ subscription/created         │ handleAppstleSubCreated  │ │
│      │ subscription/updated         │ handleAppstleSubUpdated  │ │
│      │ subscription/cancelled       │ handleAppstleCancelled   │ │
│      │ subscription/plan_changed    │ handleAppstlePlanChanged │ │
│      │ subscription/order_skipped   │ handleAppstleSkipped     │ │
│      │ billing_attempt/succeeded    │ handleBillingSuccess     │ │
│      │ billing_attempt/failed       │ handleBillingFailed      │ │
│      └──────────────────────────────┴──────────────────────────┘ │
│                                                                  │
│  5. Return 200 OK (within 5 seconds — async processing if needed)│
└──────────────────────────────────────────────────────────────────┘
```

Handler: `handleBillingSuccess` (most critical path):

```
async function handleBillingSuccess(payload) {
  // 1. Resolve customer
  const { user, customer } = await resolveCustomer(payload);

  // 2. Find or validate subscription
  const subscription = await prisma.subscription.findFirst({
    where: {
      OR: [
        { appstleContractId: payload.subscription_contract_id },
        { customerId: customer.id },
      ],
    },
  });

  if (!subscription) throw new Error("No subscription found for billing event");

  // 3. Update billing metadata
  await prisma.subscription.update({
    where: { id: subscription.id },
    data: {
      nextBillingDate: new Date(payload.next_billing_date),
      status: "active",
    },
  });

  // 4. Create shipment record
  const shipment = await prisma.subscriptionShipment.create({
    data: {
      subscriptionId: subscription.id,
      customerId: customer.id,
      shipmentDate: new Date(),
      status: "scheduled",
      assignedPrice: parseFloat(payload.amount_charged),
    },
  });

  // 5. Run assignment engine
  const assignmentResult = await runAssignment(customer, subscription, shipment);

  // 6. If auto-approved → create Shopify draft order
  if (assignmentResult.success && !assignmentResult.requiresManualReview) {
    await createShopifyDraftOrder(customer, shipment, assignmentResult.product);
  }

  // 7. If needs review → add to exception queue
  if (assignmentResult.requiresManualReview) {
    await createException(customer, assignmentResult);
  }

  return { shipmentId: shipment.id, assignment: assignmentResult };
}
```

5.2 Cabinet → Shopify (Outbound API Calls)

Action	Shopify API	When Triggered
Create Draft Order	`POST /admin/api/2024-10/draft_orders.json`	Assignment approved (auto or admin)
Complete Draft Order	`PUT /admin/api/2024-10/draft_orders/{id}/complete.json`	After draft order created (auto-complete payment)
Get Product Variants	`GET /admin/api/2024-10/products/{id}/variants.json`	Assignment engine needs variant details
Check Inventory	GraphQL `inventoryLevel` query	Assignment engine validates stock
Create Order Tag	`PUT /admin/api/2024-10/orders/{id}.json`	Tag subscription orders for tracking
Get Customer	`GET /admin/api/2024-10/customers/{id}.json`	Verify/enrich customer data

Draft Order Creation (core outbound call):

```
async function createShopifyDraftOrder(customer, shipment, product) {
  const draftOrder = await shopifyClient.rest("POST", "/draft_orders.json", {
    draft_order: {
      customer: {
        id: parseInt(customer.shopifyCustomerId),
      },
      line_items: [
        {
          variant_id: parseInt(product.shopifyVariantId),
          quantity: 1,
          price: shipment.assignedPrice.toString(),
          // Apply subscription discount if needed
          applied_discount: product.retailPrice > shipment.assignedPrice
            ? {
                title: "Subscription Discount",
                value: (product.retailPrice - shipment.assignedPrice).toFixed(2),
                value_type: "fixed_amount",
              }
            : undefined,
        },
      ],
      tags: "luciteria-subscription, cabinet-assigned",
      note: `Cabinet Assignment — Shipment ${shipment.id}`,
      use_customer_default_address: true,
    },
  });

  // Update shipment with draft order ID
  await prisma.subscriptionShipment.update({
    where: { id: shipment.id },
    data: {
      shopifyDraftOrderId: draftOrder.draft_order.id.toString(),
      status: "ordered",
    },
  });

  return draftOrder;
}
```

5.3 Assignment Engine Trigger Points

```
┌──────────────────────────────────────────────────────────────┐
│              ASSIGNMENT ENGINE TRIGGER MAP                   │
│                                                              │
│  TRIGGER                          | SOURCE       | TYPE      │
│  ─────────────────────────────────┼──────────────┼──────────  │
│  New subscription created         | Appstle hook | Automatic  │
│  Monthly billing succeeded        | Appstle hook | Automatic  │
│  Admin approves exception         | Admin UI     | Manual     │
│  Admin reassigns shipment         | Admin UI     | Manual     │
│  Product goes out of stock        | Shopify hook | Re-assign  │
│  Subscription resumed from pause  | Appstle hook | Automatic  │
│  Customer changes collection type | Appstle hook | Re-preview │
│  Bulk monthly assignment run      | Cron job     | Scheduled  │
│  (fallback for missed webhooks)   |              |            │
└──────────────────────────────────────────────────────────────┘
```

6. Database Schema Updates

6.1 Changes to Existing Models

```
// ─── Subscription (UPDATED) ──────────────────────────────────
model Subscription {
  // ... existing fields ...

  // NEW: Appstle-specific fields
  appstleContractId      String?  @unique  // Appstle's internal contract ID
  appstleSellingPlanId   String?           // Appstle selling plan GID
  appstleSellingPlanName String?           // Human-readable: "Lucite Cubes — Monthly"

  // NEW: Track billing events
  lastBillingDate        DateTime?         // When last successful charge occurred
  lastBillingAmount      Float?            // Amount of last charge
  failedBillingAttempts  Int      @default(0)  // Count of consecutive failures

  // EXISTING but now populated by Appstle:
  // shopifyContractId    — Shopify's subscription contract GID
  // status               — synced from Appstle webhook
  // nextBillingDate      — synced from Appstle webhook
  // nextShipmentDate     — calculated by Cabinet after billing success

  @@index([appstleContractId])
}

// ─── User (UPDATED) ──────────────────────────────────────────
model User {
  // ... existing fields ...

  // NEW: Link to Appstle subscription (for quick lookup)
  appstleCustomerId  String?  @unique  // Appstle's customer identifier
  shopifyCustomerId  String?  @unique  // Shopify numeric customer ID

  @@index([shopifyCustomerId])
}
```

6.2 New Models

```
// ─── AppstleWebhookLog ──────────────────────────────────
// Separate from WebhookEventLog to track Appstle-specific events
// with Appstle-specific fields and retry semantics
model AppstleWebhookLog {
  id                  String   @id @default(uuid())
  eventType           String   // "subscription/created", "billing_attempt/succeeded",
etc.
  appstleContractId   String?  // Appstle contract ID from payload
  customerEmail       String?  // Customer email from payload
  payload             String   // Full JSON payload (text for PG compat)
  status              String   @default("received") // "received", "processing", "pro-
cessed", "failed", "retrying"
  errorMsg            String?
  retryCount          Int      @default(0)
  maxRetries          Int      @default(3)
  processedAt         DateTime?
  receivedAt          DateTime @default(now())
  idempotencyKey      String?  @unique // Prevent duplicate processing

  @@index([eventType])
  @@index([status])
  @@index([appstleContractId])
  @@index([receivedAt])
  @@index([customerEmail])
}

// ─── SubscriptionTier ──────────────────────────────────
// Maps Appstle selling plans to Cabinet's assignment configuration.
// Separate from MembershipTier (which tracks store credit tiers).
model SubscriptionTier {
  id                     String   @id @default(uuid())
  name                   String   @unique  // "10mm_monthly", "25.4mm_monthly", "lu-
cite_monthly"
  displayName            String            // "10mm Cubes — Monthly"
  collectionType         String            // "10mm", "25.4mm", "lucite"
  appstleSellingPlanId   String?  @unique  // Appstle selling plan GID
  shopifySellingPlanId   String?  @unique  // Shopify selling plan GID

  monthlyPrice           Float             // $29.99, $39.99, $49.99
  billingInterval        String   @default("MONTH") // MONTH, QUARTER
  billingIntervalCount   Int      @default(1)

  // Product eligibility rules
  excludePreciousMetals  Boolean  @default(true)   // Always true for Luciteria
  maxDiscountPercent     Float    @default(0.20)    // 20% default cap
  itemsPerShipment       Int      @default(1)

  // Assignment strategy defaults
  defaultStrategy        String   @default("wishlist_priority")
  allowDuplicates        Boolean  @default(false)

  isActive               Boolean  @default(true)
  sortOrder              Int      @default(0)

  createdAt              DateTime @default(now())
  updatedAt              DateTime @updatedAt

  @@index([collectionType])
  @@index([isActive])
}

// ─── BillingEvent ──────────────────────────────────
```

```
// Ledger of all billing events from Appstle for audit trail
model BillingEvent {
  id                    String   @id @default(uuid())
  subscriptionId        String
  customerId            String
  appstleBillingId      String?  @unique  // Appstle billing attempt ID
  shopifyOrderId        String?           // Shopify order created by billing
  eventType             String            // "charge_success", "charge_failed", "re-
fund"
  amount                Float
  currency              String   @default("USD")
  billingDate           DateTime
  nextBillingDate       DateTime?
  metadata              String?           // JSON blob for extra Appstle data

  createdAt             DateTime @default(now())

  @@index([subscriptionId])
  @@index([customerId])
  @@index([billingDate])
  @@index([eventType])
}

// ─── AssignmentPreview ──────────────────────────────────────
// Pre-computed sequence of next N product assignments per subscriber.
// Recalculated on: new subscription, billing success, inventory change,
// collection type change, preference change.
model AssignmentPreview {
  id                String   @id @default(uuid())
  subscriptionId    String
  customerId        String
  sequencePosition  Int      // 1 = next shipment, 2 = month after, etc.
  productId         String?  // Assigned product (null if no eligible product)
  productSku        String?
  productTitle      String?
  estimatedDate     DateTime
  estimatedDiscount Float?   // Pre-calculated discount %
  status            String   @default("preview") // "preview", "confirmed", "shifted"
  shiftedReason     String?  // "oos", "admin_override", "preference_change"

  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  @@unique([subscriptionId, sequencePosition])
  @@index([subscriptionId])
  @@index([customerId])
  @@index([productId])
}
```

6.3 Schema Migration Summary

Change	Model	Type	Reason
Add `appstleContractId`	Subscription	Alter	Link to Appstle contract
Add `appstleSellingPlanId`	Subscription	Alter	Track which selling plan
Add `lastBillingDate/Amount`	Subscription	Alter	Billing audit
Add `failedBillingAttempts`	Subscription	Alter	Dunning tracking
Add `appstleCustomerId`	User	Alter	Appstle customer link
Add `shopifyCustomerId`	User	Alter	Direct Shopify link
Create `AppstleWebhookLog`	—	New	Appstle event audit trail
Create `SubscriptionTier`	—	New	Tier → selling plan mapping
Create `BillingEvent`	—	New	Payment ledger
Create `AssignmentPreview`	—	New	Pre-computed shipment sequence

---

7. Subscription Tiers Configuration

7.1 Tier Definitions

Tier	Collection Type	Monthly Price	Billing	Items/Shipment	Strategy
10mm Cubes	`10mm`	$29.99 (placeholder)	Monthly	1	`wish-list_priority`
25.4mm Cubes	`25.4mm`	$39.99 (placeholder)	Monthly	1	`wish-list_priority`
Lucite Cubes	`lucite`	$49.99 (placeholder)	Monthly	1	`wish-list_priority`

> **Note:** Prices are placeholders. Final pricing will be set based on average COGS per format, target margin, and competitive analysis. Each tier maps to a single Appstle selling plan.

7.2 Appstle Selling Plan Configuration

```
Selling Plan Group: "Luciteria Element Subscription"
├── Selling Plan: "10mm Cubes — Monthly"
│   ├── Billing: Every 1 month
│   ├── Price: $29.99 / month
│   ├── Delivery: Every 1 month
│   └── Metadata: { collection_type: "10mm", tier_key: "10mm_monthly" }
│
├── Selling Plan: "25.4mm Cubes — Monthly"
│   ├── Billing: Every 1 month
│   ├── Price: $39.99 / month
│   ├── Delivery: Every 1 month
│   └── Metadata: { collection_type: "25.4mm", tier_key: "25.4mm_monthly" }
│
└── Selling Plan: "Lucite Cubes — Monthly"
    ├── Billing: Every 1 month
    ├── Price: $49.99 / month
    ├── Delivery: Every 1 month
    └── Metadata: { collection_type: "lucite", tier_key: "lucite_monthly" }
```

7.3 Product Eligibility Rules

Inclusion criteria (ALL must be true):

```
Product.status          == "Active"
Product.inventoryQty    > 0
Product.format          ∈ customer's collectionType mapping
Product.availableForSubscription == true
```

Collection type → format mapping:

Collection Type	Eligible Product Formats	Eligible Categories
`10mm`	`10mm`	Metal Cube
`25.4mm`	`25.4mm`	Metal Cube
`lucite`	`50mm`	Lucite Cube

Exclusion rules (ANY triggers exclusion):

Rule	Elements Excluded	Rationale
Precious metals	Re, Rh, Au, Os, Ru, Pd, Ir, Pt	Cost exceeds subscription price; COGS unpredictable
Silver exception	Ag is **allowed**	Low enough cost for subscription
Customer already owns	Per-customer dynamic	Duplicate prevention
Previously shipped	Per-customer dynamic	Even if they no longer own it (e.g., gifted)
Admin-excluded SKUs	Per-SKU flag in `Subscription-Sku.isEligible = false`	Manual override for any reason

Eligibility check pseudocode (already implemented in assignment-engine.server.js):

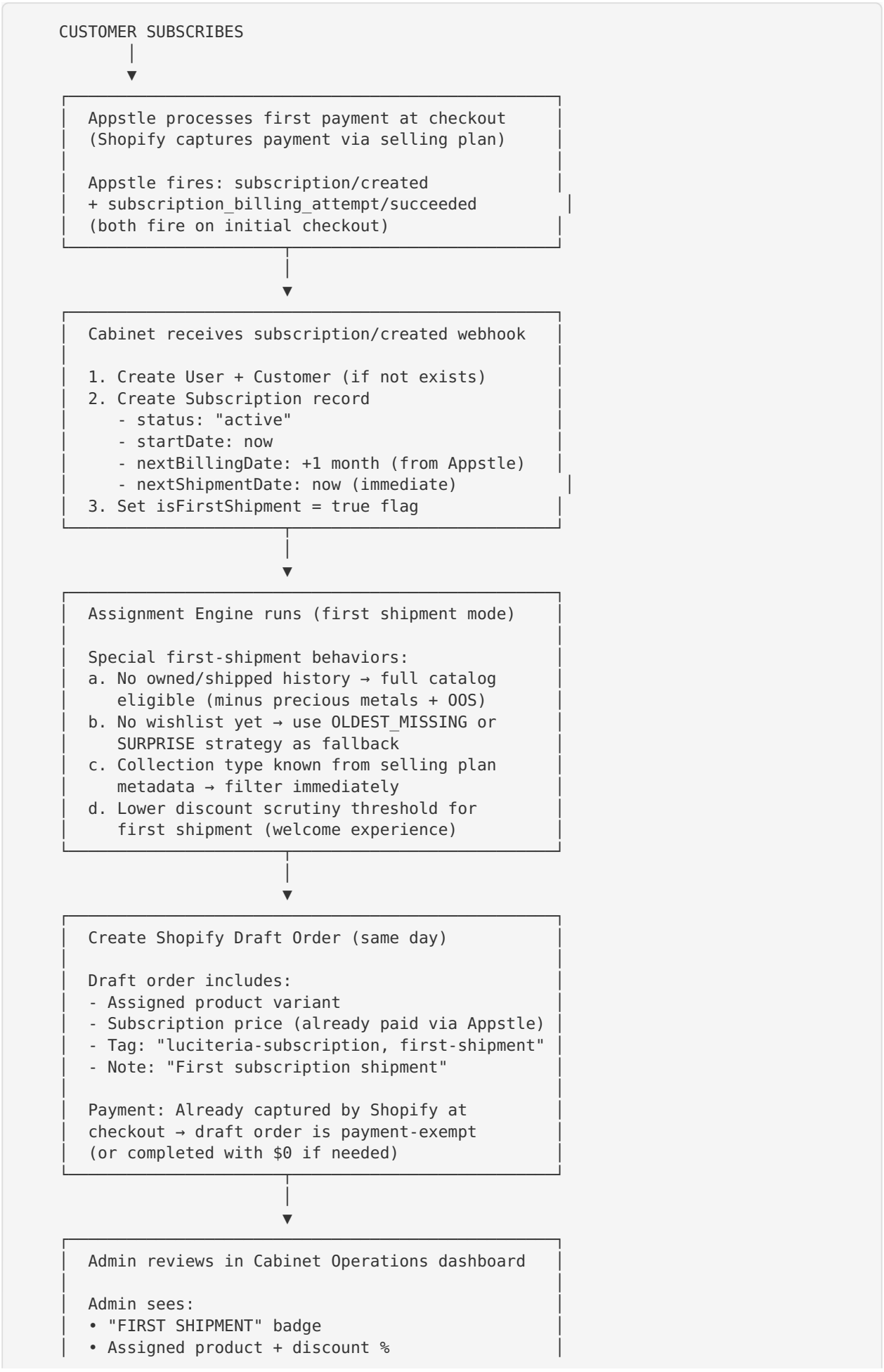
```
const PRECIOUS_METALS = ["Re", "Rh", "Au", "Os", "Ru", "Pd", "Ir", "Pt"];

function buildCandidatePool(allProducts, customer, ownedIds, shippedIds) {
  return allProducts.filter(product => {
    // Active and in stock
    if (product.status !== "Active") return false;
    if (product.inventoryQty <= 0) return false;

    // Subscription eligible
    if (!product.availableForSubscription) return false;

    // Collection type match
    const types = JSON.parse(product.collectionTypes || "[]");
    if (!types.includes(customer.collectionType)) return false;

    // Precious metal exclusion (Ag allowed)
    if (PRECIOUS_METALS.includes(product.elementSymbol)) return false;

    // Not already owned or shipped
    if (ownedIds.includes(product.id)) return false;
    if (shippedIds.includes(product.id)) return false;

    return true;
  });
}
```

8. First Shipment Logic

8.1 Problem Statement

When a customer subscribes, they expect to receive their first element **immediately** (or within typical e-commerce shipping timeframe), not wait until the next monthly billing cycle.

8.2 Flow Diagram


CUSTOMER SUBSCRIBES
Appstle processes first payment at checkout
(Shopify captures payment via selling plan)
Appstle fires: subscription/created
+ subscription_billing_attempt/succeeded
(both fire on initial checkout)
Cabinet receives subscription/created webhook
1. Create User + Customer (if not exists)
2. Create Subscription record
- status: "active"
- startDate: now
- nextBillingDate: +1 month (from Appstle)
- nextShipmentDate: now (immediate)
3. Set isFirstShipment = true flag
Assignment Engine runs (first shipment mode)
Special first-shipment behaviors:
a. No owned/shipped history → full catalog
eligible (minus precious metals + OOS)
b. No wishlist yet → use OLDEST_MISSING or
SURPRISE strategy as fallback
c. Collection type known from selling plan
metadata → filter immediately
d. Lower discount scrutiny threshold for
first shipment (welcome experience)
Create Shopify Draft Order (same day)
Draft order includes:
- Assigned product variant
- Subscription price (already paid via Appstle)
- Tag: "luciteria-subscription, first-shipment"
- Note: "First subscription shipment"
Payment: Already captured by Shopify at
checkout → draft order is payment-exempt
(or completed with $0 if needed)
Admin reviews in Cabinet Operations dashboard
Admin sees:
• "FIRST SHIPMENT" badge
• Assigned product + discount %

```
    │  • Customer details                                    │
    │  • [Approve] [Reassign] [Hold] buttons                 │
    │                                                        │
    │  On approve → Shopify fulfillment created              │
    └────────────────────────────────────────────────────────┘
```

8.3 First Shipment vs Monthly Renewal Comparison

Aspect	First Shipment	Monthly Renewal
Trigger	`subscription/created` webhook	`subscription_billing_attempt/succeeded` webhook
Payment	Captured at Shopify checkout	Captured by Appstle recurring billing
Assignment Strategy	`OLDEST_MISSING` or `SURPRISE` (no wishlist data)	`WISHLIST_PRIORITY` (preferred)
Customer History	Empty — no owned/shipped products	Built from previous shipments + collection
Discount Check	Relaxed — first experience priority	Standard 20%/35% thresholds
Shopify Order	Draft order (payment may already be captured)	Draft order → auto-complete
Admin Review	Always shown with “FIRST SHIPMENT” badge	Only if discount threshold exceeded
Timing	Same day as subscription creation	Within 24h of billing success

8.4 Edge Cases

Scenario	Handling
Customer subscribes but Cabinet is temporarily down	Webhook logged by Appstle for retry; `AppstleWebhookLog` records receipt when back up; idempotency key prevents double-processing
No eligible products for first shipment (all OOS)	Create `AssignmentException` with reason `no_eligible_items`; admin contacted; customer notified via email that first shipment is delayed
Customer already has Cabinet account with collection data	Use existing ownership data to inform first assignment — treat like a renewal
Customer subscribes to two tiers simultaneously	Each tier creates separate Subscription record; each runs independent assignment engine

---

9. Security Considerations

9.1 Webhook Verification

Appstle Webhook HMAC Validation:

Appstle signs webhooks using HMAC-SHA256 with a shared secret configured in the Appstle dashboard.

```
import crypto from "crypto";

const APPSTLE_WEBHOOK_SECRET = process.env.APPSTLE_WEBHOOK_SECRET;

export function validateAppstleWebhook(rawBody, signatureHeader) {
  if (!APPSTLE_WEBHOOK_SECRET) {
    throw new Error("APPSTLE_WEBHOOK_SECRET not configured");
  }

  const hmac = crypto.createHmac("sha256", APPSTLE_WEBHOOK_SECRET);
  hmac.update(rawBody, "utf8");
  const computed = hmac.digest("hex"); // or "base64" depending on Appstle's format

  // Timing-safe comparison to prevent timing attacks
  const sigBuffer = Buffer.from(signatureHeader, "hex");
  const compBuffer = Buffer.from(computed, "hex");

  if (sigBuffer.length !== compBuffer.length) return false;
  return crypto.timingSafeEqual(sigBuffer, compBuffer);
}
```

Shopify Webhook HMAC Validation (existing, needs production activation):

```
// Already stubbed in shopify-webhooks.server.js
// Production implementation:
export function validateShopifyWebhook(rawBody, hmacHeader) {
  const hmac = crypto.createHmac("sha256", process.env.SHOPIFY_WEBHOOK_SECRET);
  hmac.update(rawBody, "utf8");
  const computed = hmac.digest("base64");

  return crypto.timingSafeEqual(
    Buffer.from(computed, "base64"),
    Buffer.from(hmacHeader, "base64")
  );
}
```

9.2 API Authentication

Integration	Auth Method	Secret Storage
Shopify Admin API	Private app token / custom app OAuth token	`SHOPIFY_ACCESS_TOKEN` env var (Vercel)
Appstle Webhooks (inbound)	HMAC-SHA256 signature verification	`APPSTLE_WEBHOOK_SECRET` env var
Appstle API (outbound, if needed)	API key in header	`APPSTLE_API_KEY` env var
Cabinet Admin API	Session-based auth (AdminUser model)	Database (bcrypt hashed passwords)
Cabinet → Shopify	Shopify access token	`SHOPIFY_ACCESS_TOKEN` env var
Vercel deployment	Vercel project env vars	Vercel dashboard (encrypted)

9.3 Idempotency

Webhooks can be delivered multiple times (network retries, Appstle retries, infrastructure issues). Every handler must be idempotent.

```
async function processWebhookIdempotently(eventType, payload, handler) {
  // Generate idempotency key from event-specific unique fields
  const idempotencyKey = generateIdempotencyKey(eventType, payload);

  // Check if already processed
  const existing = await prisma.appstleWebhookLog.findUnique({
    where: { idempotencyKey },
  });

  if (existing?.status === "processed") {
    logger.info("Duplicate webhook — already processed", { idempotencyKey });
    return { duplicate: true, result: null };
  }

  // Log receipt
  const logEntry = await prisma.appstleWebhookLog.upsert({
    where: { idempotencyKey },
    create: {
      eventType,
      appstleContractId: payload.subscription_contract_id,
      customerEmail: payload.customer_email,
      payload: JSON.stringify(payload),
      status: "processing",
      idempotencyKey,
    },
    update: {
      status: "processing",
      retryCount: { increment: 1 },
    },
  });

  try {
    const result = await handler(payload);

    await prisma.appstleWebhookLog.update({
      where: { id: logEntry.id },
      data: { status: "processed", processedAt: new Date() },
    });

    return { duplicate: false, result };
  } catch (error) {
    await prisma.appstleWebhookLog.update({
      where: { id: logEntry.id },
      data: {
        status: logEntry.retryCount >= logEntry.maxRetries ? "failed" : "retrying",
        errorMsg: error.message,
      },
    });
    throw error;
  }
}

function generateIdempotencyKey(eventType, payload) {
  // Billing events: unique per contract + billing date
  if (eventType.includes("billing_attempt")) {
    return `${eventType}:${payload.subscription_contract_id}:${payload.billing_date}`;
  }
  // Subscription events: unique per contract + timestamp
  return `${eventType}:${payload.subscription_contract_id}:${payload.created_at || pay
load.updated_at}`;
}
```

9.4 Rate Limiting & Abuse Prevention

Protection	Implementation
Webhook endpoint rate limit	Max 100 requests/minute per IP (Vercel edge middleware)
Payload size limit	Max 1MB per webhook body
Signature required	Reject 401 immediately if HMAC header missing
IP allowlist (optional)	Appstle sends from known IP ranges — can allowlist in Vercel
Timeout	Webhook handler must respond 200 within 5 seconds; async processing for longer tasks
Dead letter queue	Failed webhooks after 3 retries → flagged for admin review in `AppstleWebhookLog`

9.5 Data Protection

Concern	Mitigation
PII in webhook logs	Store customer email for lookup; full payload stored but rotated after 90 days
Shopify tokens	Never logged, never in client-side code, only in Vercel env vars
Admin access	Separate `AdminUser` model with bcrypt passwords; session-based auth
Customer data deletion	`customers/delete` webhook archives (soft-delete) — never hard-deletes for compliance
HTTPS only	All webhook endpoints and API calls over TLS. Vercel enforces HTTPS.

9.6 Environment Variables Required

```
# Appstle Integration
APPSTLE_WEBHOOK_SECRET=        # HMAC secret for validating Appstle webhooks
APPSTLE_API_KEY=               # API key for outbound Appstle API calls (if needed)

# Shopify (existing — verify populated)
SHOPIFY_ACCESS_TOKEN=          # Shopify Admin API access token
SHOPIFY_WEBHOOK_SECRET=        # HMAC secret for Shopify webhook validation
SHOPIFY_STORE_DOMAIN=          # e.g., luciteria.myshopify.com
SHOPIFY_API_VERSION=           # e.g., 2024-10

# Database (existing)
DATABASE_URL=                  # PostgreSQL connection string

# Application
CABINET_BASE_URL=              # e.g., https://cabinet.luciteria.com
NODE_ENV=                      # production
```

Appendix A: File Map (New & Modified Files)

```
app/
├── routes/
│   ├── webhooks.appstle.jsx            ← NEW: Appstle webhook endpoint
│   ├── webhooks.inventory-update.jsx    (existing)
│   └── webhooks.product-update.jsx      (existing)
│
├── integrations/
│   ├── appstle/
│   │   ├── appstle-webhooks.server.js  ← NEW: Appstle webhook handlers
│   │   ├── appstle-client.server.js    ← NEW: Appstle API client (if needed)
│   │   └── appstle-types.js            ← NEW: TypeScript-like JSDoc type defs
│   │
│   └── shopify/
│       ├── shopify-webhooks.server.js   (existing — add subscription order handlers)
│       ├── shopify-subscriptions.server.js (existing — wire to Appstle flow)
│       └── shopify-client.server.js     (existing)
│
├── lib/
│   ├── assignment-engine.server.js      (existing — no changes needed)
│   ├── subscription-manager.server.js  ← NEW: Orchestrates assignment + order cre-
ation
│   ├── customer-resolver.server.js     ← NEW: Email-based identity matching
│   └── idempotency.server.js           ← NEW: Webhook idempotency utilities
│
└── config/
    └── subscription-tiers.server.js    ← NEW: Tier configuration constants

prisma/
├── schema.prisma                        (existing — add new models + fields)
└── migrations/
    └── YYYYMMDD_appstle_integration/   ← NEW: Migration for schema changes
```

Appendix B: Implementation Checklist

#	Task	Depends On	Priority
1	Add Appstle fields to Subscription model in Prisma schema	—	P0
2	Create `AppstleWebhookLog`, `SubscriptionTier`, `BillingEvent`, `AssignmentPreview` models	—	P0
3	Run Prisma migration	#1, #2	P0
4	Create `webhooks.appstle.jsx` route with HMAC validation	#3	P0
5	Implement `customer-resolver.server.js` (email-based matching)	#3	P0
6	Implement `handleBillingSuccess` handler (triggers assignment engine)	#4, #5	P0
7	Implement `handleAppstleSubCreated` (first shipment flow)	#6	P0
8	Implement Shopify draft order creation for assigned products	#6	P0
9	Implement `handleAppstleSubUpdated` (pause/resume/cancel)	#4	P0
10	Wire `orders/fulfilled` webhook to update CollectionRecord	—	P1

#	Task	Depends On	Priority
11	Implement `AssignmentPreview` generation and recalculation	#6	P1
12	Configure Appstle selling plans in Appstle dashboard	—	P1
13	Configure Appstle webhook URL in Appstle dashboard	#4 deployed	P1
14	Implement idempotency layer for all webhook handlers	#4	P1
15	Add webhook retry / dead letter queue monitoring	#14	P2
16	Admin UI: Show Appstle subscription details on customer page	#3	P2
17	End-to-end test: subscribe → assign → order → fulfill → collection	All above	P0